When Data Cleaning Inflates Accuracy: A Reproducible MLOps Pipeline and Leakage-Aware Evaluation for House Price Prediction

[Your Full Name] ([Roll No.])
Department of [Branch, e.g. Computer Engineering], [College Name], [City], India
[your.email@college.edu]

Term paper — [Course name and code], Semester VII, [Academic year]
Guide: [Guide's name and designation]

***Abstract*—Machine learning projects for house price prediction often report high accuracy, but preprocessing decisions taken before the train–test split can make these numbers misleading. This paper studies an open-source, end-to-end house price prediction system built with ZenML and MLflow on the Ames, Iowa housing dataset (2,930 sales). The original pipeline reported a coefficient of determination (R^2) of 0.922 for a linear regression model. We show that this figure is an artefact of its outlier-removal step, which applied a z-score rule to every numeric column of the full dataset and silently discarded 872 houses (29.8%) that have ordinary prices but rare features such as a basement half-bath or a second kitchen. Under repeated 5-fold cross-validation in which outliers are removed from the training folds only and every house is tested, the same model achieves $R^2 = -6.86 \pm 4.24$, and its mean absolute percentage error (MAPE) on houses with rare features rises to 31.5%. Restricting outlier detection to the target variable restores $R^2 = 0.868 \pm 0.033$. With a corrected, leakage-aware protocol, XGBoost using numeric and categorical features performs best ($R^2 = 0.916 \pm 0.020$, MAPE = 7.8%), although its advantage over regularised linear models is not statistically significant. We also make the pipeline portable to Windows, where ZenML's MLflow model deployer is unsupported, and measure its MLOps behaviour: identical evaluation scores across repeated runs and a median prediction latency of 66 ms.**

***Index Terms*—House price prediction, MLOps, data leakage, outlier detection, cross-validation, XGBoost, reproducibility, ZenML, MLflow.**

I. INTRODUCTION

A house is the largest asset most households own, and automated valuation models help buyers, lenders and tax assessors price it consistently. Hedonic pricing theory treats a property's price as the sum of the implicit prices of its characteristics, such as size, quality and location [1]. Machine learning extends this idea with flexible models, and the Ames, Iowa housing dataset has become a standard benchmark for it [2].

Machine learning systems are increasingly delivered as automated pipelines rather than one-off notebooks. This practice, known as machine learning operations (MLOps), addresses the hidden engineering costs of production machine learning [3], [4]. Tools such as ZenML [5] orchestrate pipeline steps and version their outputs, while MLflow [6] tracks experiments and serves models. Automation, however, does not guarantee that an evaluation is valid. Data leakage, where information that would not be available at prediction time influences training or evaluation, is common and inflates reported performance [7], [8].

This paper is a case study of an open-source house price prediction pipeline [9]. While adapting it to run on Windows, we noticed that the deployed model produced absurd prices for some ordinary houses; one typical 1976 house was valued at $1,375. We traced the problem to the pipeline's outlier-removal step, which also explains its high reported accuracy. We therefore ask three research questions:

RQ1: How much do data-cleaning choices, in particular outlier removal and where it is applied, change the reported accuracy?

RQ2: Which regression models perform best once the evaluation is free of leakage, and are the differences statistically significant?

RQ3: How well does the MLOps tooling support reproducible results and deployment across operating systems?

Our contributions are:

1) An exact reproduction of the original reported result ($R^2 = 0.9221$) and a demonstration that it is inflated because the test set was filtered; evaluated on all houses, the same model scores $R^2 = -6.86$.
2) A controlled comparison of five outlier-removal strategies under two evaluation protocols, and a step-by-step ablation from the original pipeline to a corrected one.
3) A leakage-aware comparison of five regression models with and without categorical features, using a corrected resampled t-test.
4) A Windows-compatible deployment path for the pipeline, with measurements of reproducibility, run time and prediction latency.

Section II reviews related work, Section III describes the dataset and Section IV the pipeline and evaluation method. Section V presents the results, Section VI discusses them, Section VII lists limitations and Section VIII concludes.

II. RELATED WORK

A. House Price Prediction

Hedonic regression [1] remains the conceptual basis for house price models. De Cock introduced the Ames dataset of 2,930 residential sales from 2006 to 2010 as a richer alternative to the Boston housing data, and recommended removing the few houses larger than 4,000 sq ft before modelling [2]. Sharma *et al.* compared XGBoost, support vector regression, random forest, multilayer perceptron and multiple linear regression on the same data and found XGBoost to perform best [10]. Gradient-boosted trees [11], random forests [12] and regularised linear models such as ridge [13] and lasso [14] are the standard baselines for this kind of tabular regression.

B. Data Leakage and Evaluation Pitfalls

Kaufman *et al.* formalised leakage in data mining and showed how it produces over-optimistic models [7]. Kapoor and Narayanan surveyed machine-learning-based science and found leakage affecting at least 294 papers across 17 fields; their taxonomy of eight leakage types includes preprocessing that is fitted on training and test data together [8]. When models are compared with repeated cross-validation, the training sets overlap, so the standard paired t-test is too optimistic; Nadeau and Bengio proposed a corrected variance estimate for this setting [15].

C. MLOps

Sculley *et al.* argued that most of the cost of production machine learning lies outside the model code, in data dependencies, configuration and pipelines [3]. Kreuzberger *et al.* defined MLOps through principles such as automation, versioning, reproducibility and experiment tracking [4]. MLflow implements experiment tracking, model packaging and serving [6], and ZenML composes such tools into portable pipelines [5]. These works focus on engineering; this paper shows how a pipeline can faithfully automate an evaluation that is itself flawed.

III. DATASET

The Ames Housing dataset contains 2,930 residential property sales in Ames, Iowa, between 2006 and 2010, recorded by the City Assessor's Office [2]. Each sale has 82 columns: two identifiers (Order and PID), the target SalePrice, 36 numeric predictors and 43 categorical predictors (Table I). There are 15,749 missing cells in 27 columns. In most categorical columns a missing value means that the feature is absent, for example a house without a pool or a garage.

Sale prices range from \$12,789 to \$755,000, with a median of \$160,000. The distribution is right-skewed (skewness 1.74); after the transformation $\log(1 + \text{price})$ the skewness is -0.01. All models are therefore trained to predict log price, and predictions are converted back to dollars for evaluation.

TABLE I
SUMMARY OF THE AMES HOUSING DATASET

Property	Value
Sales (rows)	2,930
Period	2006–2010
Columns	82 (2 identifiers, 1 target, 36 numeric, 43 categorical)
Missing cells	15,749 in 27 columns
Sale price: min / median / mean / max	$12,789 / $160,000 / $180,796 / $755,000
Skewness of price / of log(1 + price)	1.74 / −0.01

IV. METHODOLOGY

A. Pipeline Architecture

Fig. 1 shows the system. A ZenML training pipeline runs seven steps: (1) ingestion of the dataset from a ZIP archive; (2) filling missing numeric values with the column mean; (3) a log(1 + x) transform of living area (Gr Liv Area) and SalePrice; (4) outlier removal; (5) an 80/20 train–test split with a fixed random seed; (6) training of a scikit-learn [16] pipeline consisting of a mean imputer, one-hot encoding of categorical columns and linear regression; and (7) evaluation by mean squared error (MSE) and R^2 on the log scale. ZenML versions every step's output, and MLflow autologging records parameters, metrics and the fitted model. The code uses the factory, strategy and template-method design patterns [17], so that, for example, a different outlier or missing-value strategy can be swapped in without changing the pipeline.

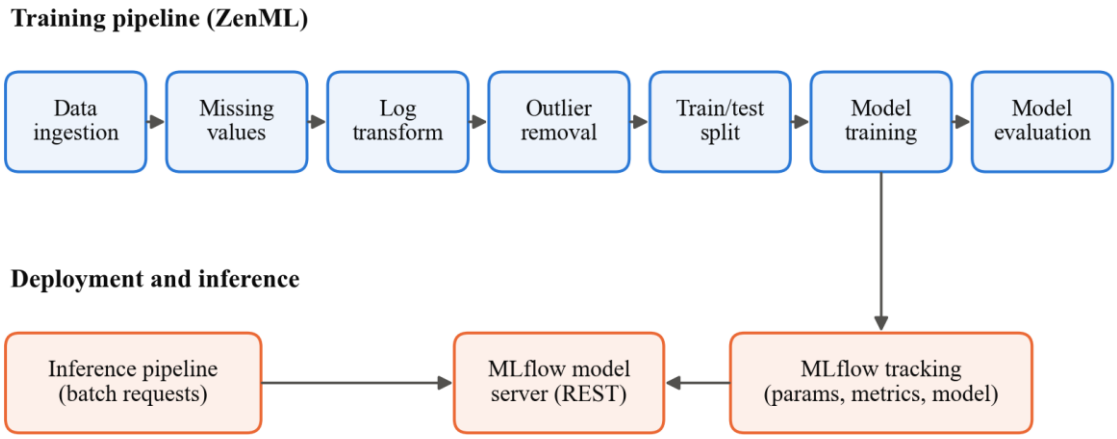


Fig. 1. Architecture of the house price prediction system. The training pipeline (top) is orchestrated by ZenML; MLflow tracks each run and serves the trained model, which a batch inference pipeline queries over HTTP.

On Linux and macOS, ZenML's MLflow model deployer starts the MLflow scoring server as a background daemon. ZenML does not support daemons on Windows, so the original deployment script fails there. We added a Windows path: the deployment script detects the operating system, retrieves the newest model from the MLflow tracking store, launches the MLflow scoring server as a detached process, waits for its health check and then runs a ZenML inference pipeline that sends batch requests over HTTP. We also replaced a hard-coded absolute data path and fixed the inference scripts, which sent raw living area to a model trained on log living area and reported the log-scale output as a price.

B. The Original Outlier Step

The original outlier step computes $z = |x - \mu| / \sigma$ for every numeric column of the full dataset, including the identifiers and the log target, and drops a row if any column has $z > 3$. Although the step is given the argument column_name = "SalePrice", it uses that argument only for input validation. The step runs before the train–

test split, so the test set is filtered too. It also keeps only numeric columns, so the model never sees categorical information such as neighbourhood or quality ratings.

This rule removed 872 of 2,930 houses (29.8%). Table II lists the columns that triggered it most often. Many numeric features are zero for most houses, so any non-zero value lies more than three standard deviations from the mean: every house with a basement half-bath, a second kitchen or a screened porch was removed. The removed houses are not price outliers; their median price was $159,000, against $161,875 for the houses that were kept.

TABLE II
COLUMNS THAT MOST OFTEN TRIGGERED THE ORIGINAL OUTLIER RULE

Column	Houses flagged	Meaning
Bsmt Half Bath	175	Half bathrooms in the basement
Kitchen AbvGr	134	Kitchens above ground (usually 2 instead of 1)
Screen Porch	107	Screened porch area (sq ft)
BsmtFin SF 2	92	Second finished basement area (sq ft)
Enclosed Porch	88	Enclosed porch area (sq ft)
Mas Vnr Area	63	Masonry veneer area (sq ft)

A house can be flagged by several columns; 872 distinct houses were removed in total.

C. Evaluation Protocol

All experiments except the reproduction in Section V-A use 5-fold cross-validation repeated 3 times with seed 42, giving J = 15 train/test splits. Every data-dependent step (imputation, scaling, encoding and outlier thresholds) is fitted on the training fold only, using scikit-learn pipelines [16]. Outlier removal is evaluated under two protocols:

Protocol A (filter before split), as in the original pipeline: the rule is applied to the whole dataset, so the test folds are filtered as well.

Protocol B (filter training folds only): the rule is fitted on and applied to the training fold, and the model is tested on every house in the test fold. This matches deployment, where the model must price any house it receives.

We report R^2 on the log scale (comparable with the original pipeline), the root mean squared error (RMSE) and mean absolute error (MAE) in US dollars, the MAPE, and the share of predictions that are off by more than 50%. Unregularised linear regression on one-hot features occasionally produced astronomically large log predictions; these were capped at |log price| = 25 so that dollar metrics remain finite, and the number of capped predictions is reported.

To compare two models we use the corrected resampled t-test of Nadeau and Bengio [15]:

$$t = \frac{\mu_d}{\sqrt{\left(\frac{1}{J} + \frac{n_2}{n_1}\right) s_d^2}} \tag{1}$$

where μ_d and s_d^2 are the mean and variance of the per-split differences in a metric between the two models, J = 15 is the number of splits and $n_2/n_1 = 1/4$ is the ratio of test to training set size. The statistic has $J - 1$ degrees of freedom, and we use a two-sided significance level of 0.05.

D. Models and Features

We compare the five models in Table III, each with two feature sets: *numeric only* (the 36 numeric predictors, identifiers removed) and *numeric + categorical* (adding the 43 categorical columns, one-hot encoded, with missing categories treated as their own level "None"; 319 features in total). Numeric features are mean-imputed and, for the linear models, standardised. Hyperparameters were fixed in advance and not tuned per fold, except for the internal cross-validation of ridge and lasso.

TABLE III
MODELS AND HYPERPARAMETERS

Model	Settings
Linear regression	Ordinary least squares
Ridge [13]	Regularisation strength chosen by efficient leave-one-out CV from 30 values in $[10^{-2}, 10^{3}]$
Lasso [14]	Regularisation strength chosen by inner 5-fold CV from 40 values
Random forest [12]	200 trees, one third of features considered per split
XGBoost [11]	1,500 trees, learning rate 0.03, depth 4, row subsample 0.8, column subsample 0.5, min. child weight 2

E. Experiments

E0 reproduces the original result on its single 80/20 split. **E1** compares five outlier strategies (none, the original global z-score, z-score on log price only, interquartile range on log price, and De Cock's rule of removing houses over 4,000 sq ft) under both protocols, using the original linear model. **E2** is an ablation that changes the original pipeline one step at a time into a corrected one. **E3** compares the models of Table III. **E4** measures permutation feature importance [12], [18] for the best model on a held-out 20% split. **E5** measures pipeline run times, run-to-run reproducibility and the latency of the deployed model server.

F. Implementation

The experiments use Python 3.11.9, scikit-learn 1.3.2, XGBoost 2.1.4, pandas 2.0.3, NumPy 1.24.4, ZenML 0.64.0 and MLflow 2.15.1, on a laptop with an Intel Core i5-1145G7 processor (8 logical cores) and 16 GB of RAM running Windows 11 Pro. E0–E4 take about 11 minutes in total.

V. RESULTS

A. Reproducing the Original Result (E0)

Our reimplementation reproduces the original pipeline's output exactly: on its single 80/20 split it keeps 2,058 houses and obtains $R^2 = 0.9221$ and MSE = 0.0109 on the log scale, identical to the values logged by the original pipeline. Changing only the outlier rule to a z-score on log price keeps 2,910 houses and gives $R^2 = 0.8568$, which again matches the corrected pipeline. This confirms that the experiments below test the same model as the original system.

B. Outlier Strategy and Evaluation Protocol (E1)

Table IV and Fig. 2 show the results. Under protocol A the original global rule appears to be the best choice, with the highest R^2 (0.910 ± 0.011) and the lowest RMSE ($19,683). Under protocol B the same rule is by far the worst: R^2 falls to -6.86 ± 4.24, the RMSE rises to $71,065 and 6.2% of predictions are off by more than 50%. The other rules are almost unaffected by the protocol and perform like no removal at all ($R^2 \approx 0.87$). Even De Cock's well-motivated rule looks better under protocol A (R^2 0.900, RMSE $22,420) than under protocol B (R^2 0.868, RMSE $49,040), because it removes the five largest and hardest-to-price houses from the test set.

TABLE IV
EFFECT OF OUTLIER STRATEGY AND EVALUATION PROTOCOL ON THE ORIGINAL LINEAR MODEL (MEAN ± SD OVER 15 SPLITS)

Outlier rule	Rows kept	Protocol	R^2 (log)	RMSE (US$)	MAPE (%)	>50% error (%)
No removal	2,930	A	0.869 ± 0.033	$40,106	10.17	0.88
No removal	2,930	B	0.869 ± 0.033	$40,106	10.17	0.88
Global Z-score (original)	2,058	A	0.910 ± 0.011	$19,683	8.11	0.55
Global Z-score (original)	2,058	B	-6.863 ± 4.245	$71,065	15.10	6.20
Target Z-score	2,910	A	0.871 ± 0.035	$39,362	9.80	0.72
Target Z-score	2,910	B	0.868 ± 0.033	$40,294	10.19	0.90

Outlier rule	Rows kept	Protocol	R^2 (log)	RMSE (US$)	MAPE (%)	>50% error (%)
Target IQR	2,870	A	0.870 ± 0.018	$34,875	9.46	0.59
Target IQR	2,870	B	0.867 ± 0.030	$39,438	10.22	0.93
De Cock (>4000 sq ft)	2,925	A	0.900 ± 0.020	$22,420	9.14	0.74
De Cock (>4000 sq ft)	2,925	B	0.868 ± 0.040	$49,040	10.06	0.85

Protocol A filters the whole dataset before splitting; protocol B filters only the training folds and tests on every house.

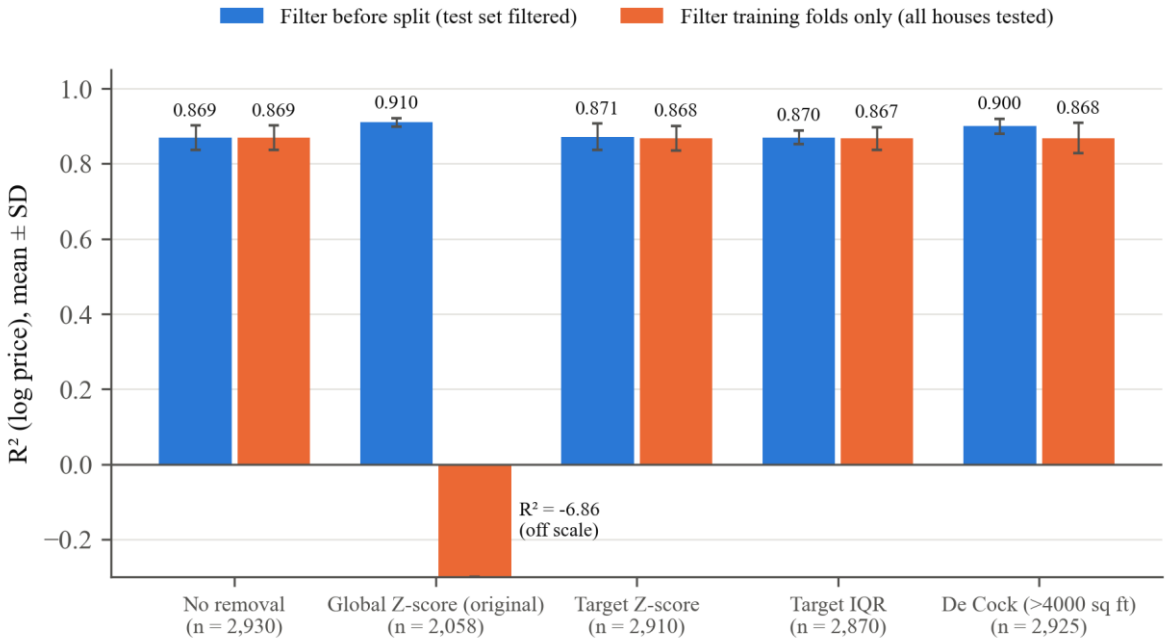


Fig. 2. R^2 of the original linear model for each outlier rule under the two protocols. The original global rule is best when the test set is filtered (blue) but fails when every house is tested (orange).

Table V explains the failure. Under protocol B we split each test fold into the 872 houses that the global rule would flag and the remaining houses. With the global rule, the model is accurate on ordinary houses (R^2 0.909, MAPE 8.1%) but fails on houses with a rare feature (R^2 −18.40, MAPE 31.5%, 19.4% of predictions off by more than 50%). In the filtered data these features are constant or almost constant: Kitchen AbvGr is 1 for every retained house, and Bsmt Half Bath is 0 except for a single mean-imputed missing value (0.061). Their coefficients are therefore estimated from almost no variation and can take extreme values. In the deployed original model, a single basement half-bath lowered the predicted log price by 4.91, i.e. multiplied the price by about 0.007, which produced the $1,375 valuation mentioned in Section I. With the target-only rule, the same houses are predicted with MAPE 14.2%.

TABLE V
ERROR ON HOUSES WITH AND WITHOUT A RARE FEATURE (PROTOCOL B)

Outlier rule	Houses	R^2 (log)	RMSE (US$)	MAPE (%)	>50% error (%)
No removal	Rare feature	0.815	$64,349	14.12	1.80
No removal	Other	0.903	$20,856	8.52	0.49
Global Z-score (original)	Rare feature	−18.395	$126,732	31.54	19.40
Global Z-score (original)	Other	0.909	$19,731	8.11	0.53
Target Z-score	Rare feature	0.811	$64,558	14.17	1.88
Target Z-score	Other	0.903	$21,193	8.51	0.49
Target IQR	Rare feature	0.813	$62,603	14.15	2.00
Target IQR	Other	0.902	$21,742	8.57	0.49
De Cock (>4000 sq ft)	Rare feature	0.805	$81,901	14.56	1.81

Outlier rule	Houses	R^2 (log)	RMSE (US$)	MAPE (%)	>50% error (%)
De Cock (>4000 sq ft)	Other	0.908	$19,751	8.18	0.45

"Rare feature" = one of the 872 houses that the original global rule removes.

C. Ablation from the Original to a Corrected Pipeline (E2)

Table VI changes the original pipeline one step at a time, evaluating each step under protocol B. Moving outlier detection to the target alone (step 2) accounts for almost the whole recovery, from R^2 −6.86 to 0.868. Fitting imputation inside the training folds (step 3) and removing the identifier columns (step 4) are correct practice but change the results only in the fourth decimal place on this dataset. Adding one-hot categorical features to unregularised linear regression (step 5) makes the model numerically unstable: with 319 features, some of them rare categories, the least-squares problem is ill-conditioned, and on average 6 test predictions per split hit the cap. Adding ridge regularisation (step 6) resolves this and gives the best linear result, R^2 = 0.900 ± 0.026, RMSE $30,777 and MAPE 8.47%.

TABLE VI
ABLATION FROM THE ORIGINAL PIPELINE TO A CORRECTED PIPELINE

Step	Change	R^2 (log)	RMSE (US$)	MAPE (%)
0	Original, as reported (single split, test set filtered)	0.922	n/a	n/a
1	Original pipeline, honest test set	−6.863 ± 4.245	$71,065	15.10
2	+ Outlier rule on SalePrice only	0.868 ± 0.033	$40,294	10.19
3	+ Imputation fitted on training folds	0.868 ± 0.033	$40,287	10.19
4	+ Drop ID columns (Order, PID)	0.867 ± 0.033	$40,500	10.20
5	+ Categorical features (one-hot)	−50.818 ± 37.792	≈ $4.7 × 10^9	≈ 2.6 × 10^5
6	+ Ridge regularisation	0.900 ± 0.026	$30,777	8.47

Steps 1–6 use protocol B with 15 splits (mean ± SD); each step keeps all previous changes.

D. Model Comparison (E3)

Table VII and Fig. 3 compare the models under the corrected protocol. XGBoost with numeric and categorical features is best on every metric: R^2 = 0.916 ± 0.020, RMSE $22,686, MAE $13,156 and MAPE 7.77%. Categorical features improve every model except unregularised linear regression; for ridge, R^2 rises from 0.866 to 0.900, and for XGBoost from 0.900 to 0.916. The tree ensembles also have far more stable errors across splits (RMSE standard deviation about $3,000) than the linear models ($13,000–$24,000), because linear models extrapolate badly on a few very large or unusual houses, and those houses dominate the RMSE of the splits that contain them.

TABLE VII
MODEL COMPARISON UNDER THE CORRECTED PROTOCOL (MEAN ± SD OVER 15 SPLITS)

Model	Features	R^2 (log)	RMSE (US$)	MAE (US$)	MAPE (%)	Fit time (s)
Linear Regression	Numeric	0.867 ± 0.033	$40,500	$17,693	10.20	0.07
Ridge	Numeric	0.866 ± 0.034	$42,035	$17,715	10.23	0.13
Lasso	Numeric	0.868 ± 0.031	$39,139	$17,633	10.21	0.28
Random Forest	Numeric	0.880 ± 0.026	$27,386	$16,079	9.52	6.07
XGBoost	Numeric	0.900 ± 0.022	$24,688	$14,768	8.74	6.49
Linear Regression	Num. + cat.	−50.818 ± 37.792	≈ $4.7 × 10^9	≈ $3.3 × 10^8	≈ 2.6 × 10^5	0.54
Ridge	Num. + cat.	0.900 ± 0.026	$30,777	$14,475	8.47	1.13
Lasso	Num. + cat.	0.899 ± 0.027	$30,884	$14,337	8.45	0.91
Random Forest	Num. + cat.	0.887 ± 0.025	$26,550	$15,532	9.21	13.70
XGBoost	Num. + cat.	0.916 ± 0.020	$22,686	$13,156	7.77	8.63

Fit time is per split on the hardware in Section IV-F. Linear regression with categorical features was numerically unstable (Section V-C).



Fig. 3. Mean RMSE in US dollars of each model with numeric features only (blue) and with numeric and categorical features (orange). Lower is better.

Table VIII tests whether the best model's advantage is statistically significant. XGBoost with categorical features is significantly better than every numeric-only model on log RMSE and MAE ($p \leq 0.012$) and than both random forests on all three metrics ($p < 0.001$). Its advantage over ridge and lasso with categorical features is not significant (log RMSE $p = 0.27$ and 0.26; MAE $p = 0.056$ and 0.095). A regularised linear model with categorical features is therefore a strong, fast and interpretable alternative: it fits in about one second per split, compared with 8.6 s for XGBoost.

TABLE VIII
CORRECTED RESAMPLED T-TEST: XGBOOST (NUM. + CAT.) AGAINST EACH OTHER CONFIGURATION

Model	Features	Δ RMSE (log)	p	Δ MAE (US$)	p	Δ RMSE (US$)	p
Linear Regression	Numeric	0.0298	0.011	$4,537	< 0.001	$17,814	0.179
Ridge	Numeric	0.0306	0.012	$4,560	< 0.001	$19,349	0.177
Lasso	Numeric	0.0296	0.007	$4,477	< 0.001	$16,454	0.175
Random Forest	Numeric	0.0228	< 0.001	$2,923	< 0.001	$4,700	< 0.001
XGBoost	Numeric	0.0109	< 0.001	$1,612	< 0.001	$2,002	< 0.001
Linear Regression	Num. + cat.	2.6660	< 0.001	≈ $3.3 × 10⁸	0.020	≈ $4.7 × 10⁹	< 0.001
Ridge	Num. + cat.	0.0106	0.267	$1,319	0.056	$8,091	0.330
Lasso	Num. + cat.	0.0108	0.262	$1,182	0.095	$8,199	0.303
Random Forest	Num. + cat.	0.0192	< 0.001	$2,376	< 0.001	$3,865	< 0.001

Δ = other model minus XGBoost (positive means XGBoost has the lower error). Two-sided p-values, 14 degrees of freedom.

E. Prediction Quality and Feature Importance (E4)

On a held-out 20% split (586 houses), the best model reaches $R^2 = 0.948$ on the log scale and 0.930 in dollars, with RMSE $23,705, MAE $13,237 and MAPE 6.76%. Fig. 4 plots cross-validated predictions against actual prices. Predictions follow the diagonal closely up to about $400,000; above that the model tends to under-predict, since only 17 houses in the dataset sold for more than $500,000.

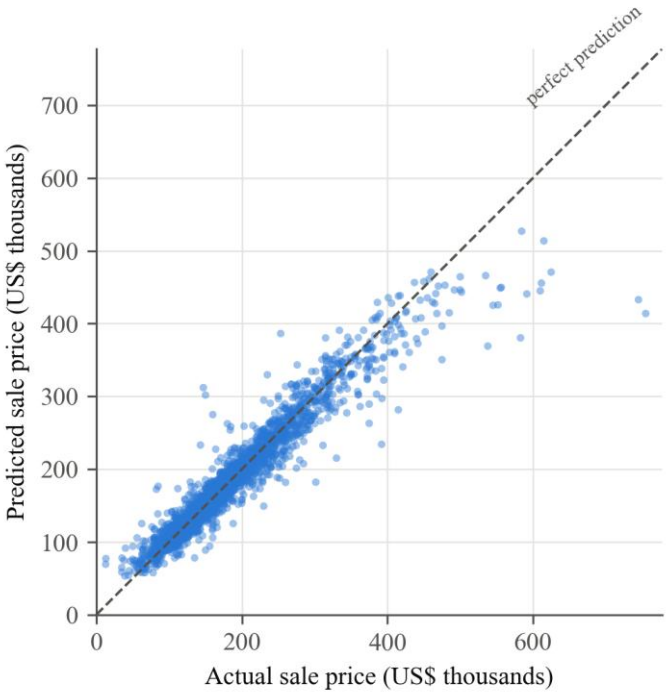


Fig. 4. Cross-validated predictions of XGBoost (numeric + categorical) against actual sale prices; each house is predicted once by a model that did not see it during training.

Fig. 5 shows permutation importance, i.e. how much the log RMSE increases when one feature's values are shuffled. Above-ground living area (0.076) and overall quality (0.062) dominate, followed by year built, overall condition and basement area. Neighbourhood is the most important categorical feature. This ranking agrees with hedonic theory, where size, quality, age and location drive price [1]. Correlated features, such as living area and first-floor area, share importance, so individual values should be read with care [18].

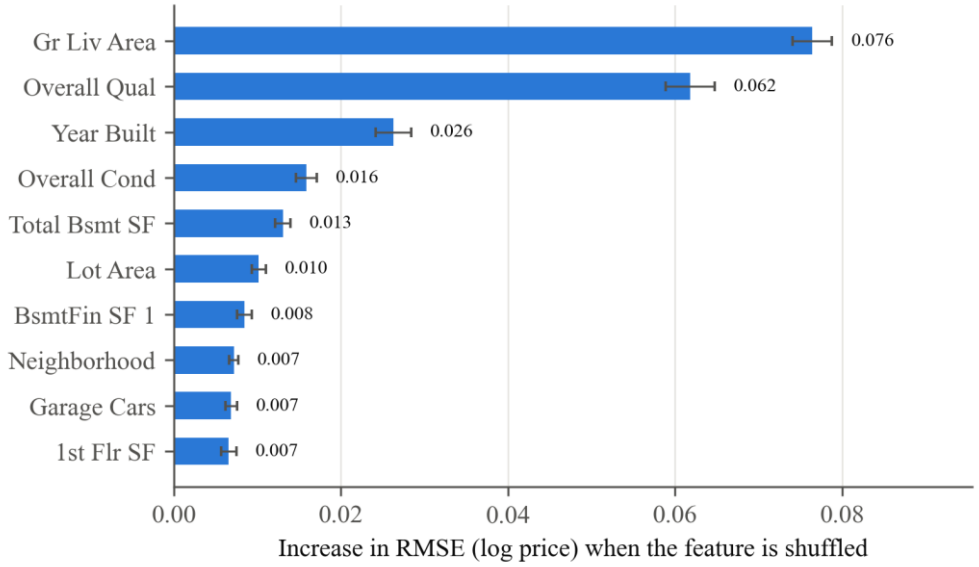


Fig. 5. The ten most important features of the best model by permutation importance on the held-out split (mean ± SD over 10 shuffles).

F. MLOps Behaviour (E5)

Table IX summarises the MLOps measurements. Training runs that used the same code produced identical evaluation scores to all 16 printed decimal places: the 2 runs of the original pipeline gave R^2 = 0.9221139768594216, and the 3 runs after the outlier fix gave 0.8567861675345597. The deployed model

server answers a single-house request in a median of 66.1 ms, and a 100-house batch in 68.3 ms. Latency is therefore dominated by HTTP and framework overhead rather than by the model, and batching reduces the cost per house to under 1 ms. These latency figures are for the linear model that the ZenML pipeline deploys. Fig. 6 shows the MLflow dashboard used to track the runs.

TABLE IX
MLOPS MEASUREMENTS ON THE WINDOWS LAPTOP

Measurement	Value
Completed training pipeline runs	5
Training pipeline duration: median (min–max)	54.0 s (37.0–83.7 s)
Distinct R^2 values across runs with the same code	1 per code version (identical to 16 decimal places)
Inference pipeline duration (median of 4 runs)	5.6 s
Model server cold start	39.8 s
Latency, 1 house per request (200 requests): median / p95	66.1 / 78.0 ms
Latency, 100 houses per request (50 requests): median / p95	68.3 / 78.1 ms

[Insert a screenshot of the MLflow dashboard here: double-click 4_open_mlflow_dashboard.bat, open the ml_pipeline experiment and capture the list of runs]

Fig. 6. MLflow tracking dashboard showing the training runs of the ml_pipeline experiment.

VI. DISCUSSION

A. Why Preprocessing Inflated the Accuracy

The original pipeline's R^2 of 0.922 was not wrong arithmetic; it described a different population from the one the model would serve. Removing rows before the split changes the test set, so the metric no longer measures performance on the houses the model will be asked to price. The z-score rule made this worse because it assumes roughly normal data, while many housing features are zero for most houses; on such features every non-zero value is an "outlier". The result was a model that looked best in evaluation and failed on almost a third of real houses. Kapoor and Narayanan list preprocessing on training and test data together as a common leakage type [8]; our results show that it can change not only the size of a reported metric but also which modelling choice looks best.

From these results we suggest five practical checks for similar projects:

1) Fit every data-dependent step, including outlier thresholds, on training data only, ideally inside a single pipeline object.
2) Never filter the test set unless the same filter will run in production and the excluded cases are handled separately.
3) Choose outlier rules per variable with domain reasoning, and log how many rows each step removes.
4) Report errors in the original units and for subgroups, not only R^2 on a transformed scale.
5) Inspect individual predictions for plausibility before trusting an aggregate metric.

B. Choosing a Model

Under a fair protocol, categorical information and regularisation matter more than the choice of algorithm family. XGBoost gave the lowest errors and the most stable results across splits, but ridge and lasso with categorical features were not significantly worse on log RMSE, and they train about eight times faster and are easier to explain. The right choice depends on whether accuracy, speed or interpretability matters most.

C. Lessons for MLOps

The MLOps tooling made the problem easy to diagnose: versioned artefacts and tracked runs allowed an exact reproduction of the original score, and repeated runs were deterministic. However, a pipeline automates whatever evaluation it encodes, and here it reproduced a misleading score reliably. Automated checks on data volume, such as an alert when a cleaning step removes more than a few percent of rows, would have caught the problem early. Portability is a second practical issue: a deployment component that silently assumes Unix daemons prevents the pipeline from running on Windows machines, which are common in teaching and enterprise environments.

VII. LIMITATIONS AND THREATS TO VALIDITY

This study uses a single dataset from one city and the period 2006–2010, which includes the 2008 financial crisis; results may differ for other markets. Random cross-validation ignores time, so a model trained on later sales and tested on earlier ones is included; a time-based split would be more realistic. Hyperparameters were fixed rather than tuned, which may disadvantage the tree ensembles. Cross-validation splits overlap; the corrected t-test reduces but does not remove this dependence, and the nine comparisons in Table VIII were not corrected for multiple testing. Permutation importance can be misleading for correlated features. The MLOps timings come from one laptop and will vary with hardware. Finally, predictions of the unstable linear model were capped to keep dollar metrics finite.

VIII. CONCLUSION AND FUTURE WORK

We examined an open-source MLOps pipeline for house price prediction and showed that its reported R^2 of 0.922 was produced by an outlier step that removed 29.8% of houses from both training and test data. Evaluated on all houses, the same model scored $R^2 = -6.86$; limiting outlier detection to the target restored $R^2 = 0.868$. Under a leakage-aware protocol, XGBoost with numeric and categorical features performed best ($R^2 = 0.916$, MAPE = 7.8%), with regularised linear models statistically close behind. We also made the pipeline deployable on Windows and showed that its runs are exactly reproducible.

Future work includes time-based validation, hyperparameter tuning as a pipeline step, deploying the best model through the pipeline, prediction intervals that quantify uncertainty, and automated data checks and drift monitoring in production.

ACKNOWLEDGMENT

The author thanks [Guide's name] for guidance. The baseline pipeline was adapted from the open-source project in [9]; Section IV-A lists the changes made to it. [If your institution requires it, state here how AI tools were used in preparing this work.]

REFERENCES

[1] S. Rosen, "Hedonic prices and implicit markets: Product differentiation in pure competition," *Journal of Political Economy*, vol. 82, no. 1, pp. 34–55, 1974.

[2] D. De Cock, "Ames, Iowa: Alternative to the Boston housing data as an end of semester regression project," *Journal of Statistics Education*, vol. 19, no. 3, 2011, doi: 10.1080/10691898.2011.11889627.

[3] D. Sculley *et al.*, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems 28 (NeurIPS)*, 2015, pp. 2503–2511.

[4] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31866–31879, 2023, doi: 10.1109/ACCESS.2023.3262138.

[5] ZenML, "ZenML: An open-source MLOps framework," version 0.64.0, 2024. [Online]. Available: https://zenml.io

[6] M. Zaharia *et al.*, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.

[7] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, Art. 15, 2012.

[8] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, Art. 100804, 2023, doi: 10.1016/j.patter.2023.100804.

[9] A. Singh, "prices-predictor-system: An end-to-end house price prediction pipeline with ZenML and MLflow," source code. [Online]. Available: [insert the URL you downloaded the project from]

[10] H. Sharma, H. Harsora, and B. Ogunleye, "An optimal house price prediction algorithm: XGBoost," *Analytics*, vol. 3, no. 1, pp. 30–45, 2024.

[11] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[12] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[13] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[14] R. Tibshirani, "Regression shrinkage and selection via the lasso," *Journal of the Royal Statistical Society: Series B*, vol. 58, no. 1, pp. 267–288, 1996.

[15] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003.

[16] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[17] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

[18] A. Fisher, C. Rudin, and F. Dominici, "All models are wrong, but many are useful: Learning a variable's importance by studying an entire class of prediction models simultaneously," *Journal of Machine Learning Research*, vol. 20, no. 177, pp. 1–81, 2019.

APPENDIX: REPRODUCING THE RESULTS

All results can be regenerated on Windows from the project folder. Run setup_windows.bat once, then:

```
venv\Scripts\python.exe research\experiments.py  (E0–E4, about 11 minutes)
venv\Scripts\python.exe research\significance_all_metrics.py  (Table VIII)
venv\Scripts\python.exe research\mlops_measurements.py  (E5, Table IX)
venv\Scripts\python.exe research\make_figures.py  (Figs. 1–5)
venv\Scripts\python.exe run_deployment.py  (train, deploy and run batch inference)
```

Results are written to research/results and figures to research/figures.